

Ensemble Methods: Boosting vs. Bagging

Final Project Report

Nadir Askarov

AI Academy, National AI Center

Spring 2026

nadir.eskerov25@aiacademy.az

Abstract—We compare boosting and bagging empirically, implementing a CART decision tree, AdaBoost (discrete SAMME over stumps) and a Random Forest from scratch in NumPy and validating them against scikit-learn, which they match to within 0.6% on our two larger datasets. Across three datasets chosen to differ in dimension, class balance and class overlap, the two ensembles prove far closer than the question suggests - on Adult they differ by 0.0005 accuracy, well inside fold-to-fold variation - but they fail in opposite and predictable ways. A bootstrap decomposition of the 0/1 loss isolates the mechanism: bagging leaves the base learner’s bias exactly unchanged (0.0265 for both a single tree and the forest) while cutting variance by 64%, whereas AdaBoost is the only method that reduces bias, by 33%. This predicts the rest. AdaBoost wins on clean, balanced, low-dimensional data (0.9754 vs 0.9595 on Breast Cancer), loses badly under 20% label noise (giving up 6.2 accuracy points against the forest’s 0.9), and falls below even a single decision tree on 7-class Covertype, where a stump is too weak to boost. An unsupervised pipeline (PCA, K-Means, DBSCAN) ranks the datasets by K-Means ARI in the same order the classifiers rank them by accuracy, though we argue the relationship is one-directional.

I. INTRODUCTION

Boosting and bagging both build a strong classifier out of many weak ones, but they disagree about what makes a classifier weak in the first place. Bagging assumes the base learner is unstable: a fully grown tree fits its sample so closely that resampling the data changes the tree completely, and averaging many such trees cancels that instability out. Boosting assumes the opposite problem. Its base learner is a stump, far too rigid to fit anything interesting, so the ensemble spends its rounds correcting the stump’s systematic mistakes rather than averaging away its noise. The two therefore attack different halves of the same error.

This paper asks the question the brief poses: *under what conditions does boosting outperform bagging, and vice versa, and why?* Rather than take the textbook answer on faith, we implemented a CART decision tree, AdaBoost (discrete SAMME) over stumps, and a Random Forest from scratch in NumPy, and compared them on three datasets chosen to differ in what theory says should matter: dimension, class balance, and class overlap.

Our headline result is that on clean, well-separated data the two are close enough that the difference is not worth arguing about, but the gap opens under label noise, in the direction the bias-variance decomposition predicts. AdaBoost’s

re-weighting rule cannot distinguish a hard example from a mislabelled one, so it spends its later rounds fitting corrupted points; bagging averages them away. The datasets where the ensembles help least also turn out to be those where our unsupervised analysis finds no class structure along the data’s dominant directions.

Section II reviews the background, Section III our implementations, Section IV the setup, Section V the seven experiments, and Section VI interprets them.

II. BACKGROUND

A. Decision Trees

A decision tree partitions the feature space by repeatedly choosing the split that most reduces an impurity measure [1]. We follow CART, which for a node with class proportions p_c uses either the Gini index $I_G(p) = 1 - \sum_{c=1}^C p_c^2$ or the entropy $I_E(p) = -\sum_{c=1}^C p_c \log_2 p_c$, and scores a candidate split by the weighted impurity drop

$$\Delta I = I(\text{parent}) - \frac{N_L}{N} I(\text{left}) - \frac{N_R}{N} I(\text{right}). \quad (1)$$

Gini and entropy rank splits almost identically; Gini is cheaper because it avoids the logarithm, so we made it the default. The property that matters below is that an unpruned tree has low bias and high variance: it can express nearly any boundary, but which one depends heavily on the sample it saw. Quinlan’s ID3 [2] is the other classical line, using information gain on categorical attributes.

B. Boosting

AdaBoost [3] maintains a weight w_i on each training example, initialised to $1/N$. At round m it fits a weak learner h_m to the weighted sample, measures the weighted error

$$\epsilon_m = \frac{\sum_{i=1}^N w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i]}{\sum_{i=1}^N w_i^{(m)}}, \quad (2)$$

and gives the learner a vote

$$\alpha_m = \ln \frac{1 - \epsilon_m}{\epsilon_m} + \ln(K - 1), \quad (3)$$

where K is the number of classes; the $\ln(K - 1)$ term is the SAMME generalisation [1] and vanishes for $K = 2$. Weights are then updated multiplicatively,

$$w_i^{(m+1)} \propto w_i^{(m)} \exp(\alpha_m \mathbf{1}[h_m(x_i) \neq y_i]), \quad (4)$$

and renormalised, so misclassified points carry more weight into the next round. The final prediction is $\hat{y} = \arg \max_k \sum_m \alpha_m \mathbf{1}[h_m(x) = k]$. Equations (3) and (4) are the whole mechanism, and they explain both of AdaBoost’s characteristic behaviours. Since α_m grows without bound as $\epsilon_m \rightarrow 0$, even a learner barely better than chance contributes, which is how a sequence of stumps drives training error down - a reduction in bias. And since the update looks only at whether a point was misclassified, a mislabelled point is indistinguishable from a genuinely hard one, and its weight grows every round it is missed. That is the mechanism behind the noise sensitivity we measure below.

C. Bagging and Random Forests

Bagging [4] trains each member on a bootstrap replicate of the training set and aggregates by voting. Since each replicate omits roughly $1/e \approx 37\%$ of the data, averaging over replicates reduces the variance of the aggregate without touching its bias. Random Forests [5] add a second source of randomness: at every node only a random subset of features (commonly $\lfloor \sqrt{p} \rfloor$) is considered. This matters because averaging T identically distributed but correlated predictors with pairwise correlation ρ leaves variance $\rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$ - the second term vanishes as T grows, but the first does not, so decorrelating the trees is what makes the average keep improving. It also gives a free validation estimate: each point is out-of-bag (OOB) for about a third of the trees, and voting only those trees yields the OOB score we report alongside test accuracy.

D. Unsupervised Techniques

PCA [6] projects onto the eigenvectors of the covariance matrix with the largest eigenvalues, i.e. the directions of greatest variance. K-Means [7] alternates assigning points to the nearest of k centroids and recomputing those centroids, minimising within-cluster sum of squares. It assumes clusters are roughly spherical and comparably sized, and it labels every point. DBSCAN [8] instead grows clusters from points with at least `min_samples` neighbours within ϵ , which lets it find arbitrarily shaped clusters and, unlike K-Means, refuse to assign outliers by marking them as noise. The distinction that matters for our analysis is that K-Means partitions on distance to a centre while DBSCAN partitions on local density, so they disagree exactly where clusters touch.

III. METHODS

Everything below is implemented from scratch in NumPy. scikit-learn appears in exactly two places, both permitted by the brief: `DecisionTreeClassifier` and `RandomForestClassifier` as correctness baselines, and `adjusted_rand_score` for ARI. The metrics, scaler and splitter are ours; no `sklearn.ensemble` class is used as an implementation. Code: <https://github.com/nadir2609/Ensemble-Methods> (tag `v1.0-final`).

A. Decision Tree

The tree follows CART with an exhaustive split search over continuous features. For each candidate feature we sort the column once, then sweep thresholds at the midpoints of consecutive distinct values, scoring each by equation (1) and accumulating weighted class counts with `cumsum`, so every threshold for a feature is scored in one vectorised pass rather than by rescanning the node’s samples. That is the difference between $O(N \log N)$ and $O(N^2)$ per feature, and it is what makes the 100-tree forests below finish at all. Splitting stops on `max_depth`, `min_samples_split`, a pure node, or no split giving a positive impurity drop.

Two details matter. `fit` takes an optional `sample_weight`, and `impurity` uses weighted class proportions $p_c = \sum_{i:y_i=c} w_i / \sum_i w_i$ throughout - this is what AdaBoost needs, and a uniform weight vector recovers the ordinary tree exactly. And `max_features` draws a random feature subset per node from the tree’s seeded generator, which is what the forest uses to decorrelate its members. One class therefore serves all three roles.

B. AdaBoost

Discrete SAMME with depth-1 trees, following equations (2)-(4). Each round’s stump is seeded from `random_state + m` for reproducibility.

The brief leaves two choices to us. When $\epsilon_m = 0$ we clip to 10^{-10} rather than divide by zero; a perfect stump ends boosting anyway by driving all remaining weights to zero. When a learner is no better than random we stop rather than reset the weights, using the SAMME threshold $\epsilon_m \geq 1 - 1/K$ rather than the binary $\epsilon_m \geq 0.5$: with K classes random guessing gets $1 - 1/K$ wrong, so a stump on 7-class Covertype can sit well above 0.5 error and still be useful, and the binary threshold would have halted boosting after a handful of rounds. We stop rather than reset because a truncated ensemble is honest about having run out of signal. `predict_proba` normalises the per-class sums of α_m ; `staged_predict` yields the running prediction after each round, producing the curves in Experiment 2.

C. Random Forest

Each tree is fitted on a size- N bootstrap sample with `max_features="sqrt"` and `random_state` derived from the forest’s seed and tree index. We record OOB indices at fit time, so the OOB score costs one extra pass: for each sample we aggregate only the trees that did not see it and take the majority vote. Samples in-bag for every tree are excluded rather than counted as errors. `predict` is a majority vote; `predict_proba` averages the trees’ probabilities.

D. Unsupervised Pipeline

PCA centres the data and eigen-decomposes the covariance matrix with `numpy.linalg.eigh`, valid because covariance is symmetric; eigenvalues return ascending, so we reverse and keep the leading components. **K-Means** is Lloyd’s algorithm with `k-means++` seeding, stopping when the centroid

TABLE I
DATASETS USED IN THE STUDY. SIZES ARE AFTER SUBSAMPLING AND,
FOR ADULT, AFTER ONE-HOT ENCODING.

Dataset	Samples	Features	Classes	Minority
Breast Cancer	569	30	2	37.3%
Adult Income	8 000	104	2	24.9%
Covertypes	9 999	54	7	0.47%

shift falls below $\tau_{0.1}$. We use k-means++ because uniform seeding on standardised Covertypes landed in obviously bad local minima; even so, every reported k takes the best of 10 restarts by inertia. **DBSCAN** uses brute-force region queries and grows clusters from core points with a breadth-first frontier, re-labelling provisional noise as a border point when later reached from a core point. Brute force is $O(N^2)$ in distances, which is why the unsupervised experiments cap at 2,000 samples.

We cluster in the retained PCA subspace rather than raw feature space. This mainly helps DBSCAN: density estimates degrade as dimension grows because distances concentrate, and on Covertypes’s 54 raw features the ε sweep gave either one giant cluster or all noise, with nothing in between.

IV. EXPERIMENTAL SETUP

A. Datasets

The three datasets in Table I were chosen to vary the properties the bias-variance argument says should decide the comparison, rather than to collect three datasets that look alike.

Breast Cancer Wisconsin is small, balanced, binary and close to linearly separable - the control, and the dataset for the bias-variance decomposition, which needs a balanced binary problem. **Adult Income** is larger, moderately imbalanced, and mixes continuous and categorical attributes; one-hot encoding turns 14 raw columns into 104 features, mostly sparse binary indicators, a genuinely different regime for a tree than 30 dense continuous measurements. It meets the > 20 feature requirement, as does Breast Cancer at 30. **Covertypes** supplies multi-class structure (7 cover types) and the severe imbalance the brief requires: Cottonwood/Willow is 0.47% of the data, 47 of 9,999 rows. Our class-stratified draw from the full 581,012-row file preserves each class’s proportion and never drops a class, so the imbalance survives subsampling.

Subsampling deserves a note. Our trees are exact but much slower than sklearn’s C implementation, so Adult is capped at 8,000 rows at load time and the repeated-fit sweeps (Experiments 2, 3, 5) cap training at 2,000; baselines use the full loaded data, and every cap is stratified. This depresses absolute accuracies - these are not state-of-the-art numbers and are not meant to be - but it applies identically to every model, so the comparison between them is unaffected.

B. Preprocessing

Rows carrying Adult’s ? marker are dropped rather than imputed. That is defensible here because the affected columns are categorical (*workclass*, *occupation*,

native_country), where imputing a mode would invent a category membership the tree would then split on, and because it costs 7.4% of rows. We would not make the same call on a smaller dataset.

Features are standardised to zero mean and unit variance, with the scaler fitted on training data only - inside each CV fold, not once globally, so no test statistics leak into training. Trees do not need this, being invariant to monotone rescaling, but PCA, K-Means and DBSCAN measure distances and would otherwise be dominated by whichever feature has the largest units.

For Covertypes’s imbalance we rely on the stratification running through every split, cap and fold, and report macro-F1 alongside accuracy so that ignoring a rare class is visibly punished - a classifier that never predicts Cottonwood/Willow still scores 99.5% accuracy. We implemented random oversampling and inverse-frequency class weights (*src/utis/preprocessing.py*), but report headline results without resampling so all models are compared on identical data; Section VI returns to this.

C. Metrics

We report accuracy, macro-F1 (equal weight per class, which is what we want on Covertypes), and ROC-AUC - positive-class probability for binary, macro one-vs-rest for multi-class. Our AUC is a rank-based estimator handling tied scores properly: ties matter because a stump emits few distinct probability values, and treating tied scores as ordered inflates AUC. All randomness is seeded from `random_state=42`.

D. Experiments

All seven are reproduced end to end by `python experiments/run_all.py`, which regenerates every figure in this paper from scratch.

- 1) **Baseline.** Unpruned tree, a depth-1 stump, and sklearn’s `DecisionTreeClassifier` with matching parameters, on an 80/20 stratified split. This is the correctness check: our tree must land within 2% of sklearn’s accuracy.
- 2) **AdaBoost scaling.** One 200-stump fit per dataset, with `staged_predict` giving train and test error after every round, to see whether and when boosting overfits.
- 3) **Random Forest scaling.** Test and OOB accuracy as trees are added up to 200 with `max_depth=None`; then test accuracy for a 50-tree forest at depths 1-20.
- 4) **Head-to-head.** 100 estimators each, 5-fold stratified CV, reporting mean $\pm$ std of accuracy, macro-F1 and AUC.
- 5) **Noise robustness.** Flip $\eta \in \{0, 5, 10, 20\}\%$ of *training* labels, evaluate on the clean test set.
- 6) **Bias-variance decomposition.** 100 bootstrap replicates on Breast Cancer, decomposed per Breiman [4].
- 7) **Unsupervised analysis.** Scree, elbow and k -distance plots; PCA scatters coloured by true, K-Means and DBSCAN labels; ARI throughout.

TABLE II
EXPERIMENT 1. TEST ACCURACY ON AN 80/20 STRATIFIED SPLIT. Δ IS $|\text{ACC}(\text{OURS}) - \text{ACC}(\text{SKLEARN})|$.

Dataset	Ours	Stump	sklearn	Δ
Breast Cancer	0.9292	0.8850	0.9027	0.0265
Adult	0.8137	0.7512	0.8075	0.0062
Covertypes	0.7120	0.6480	0.7155	0.0035

V. RESULTS

A. Baseline and correctness

Table II compares our tree against sklearn’s with matching parameters. On Adult (0.62 points apart) and Covertypes (0.35) we are comfortably inside the 2% tolerance the brief asks for.

Breast Cancer misses it at 2.65%, and we would rather explain that than tune the seed until it disappears. The discrepancy is in our favour (0.9292 vs 0.9027) and the test split holds 114 samples, so 2.65% is three test points - one tie broken differently deep in the tree moves the number by nearly a percent, which makes a 2% threshold on 114 samples a blunt instrument. The implementations do break ties differently: where several thresholds give an identical impurity drop, common here because the features are strongly correlated, ours takes the first in `argmax` order while sklearn draws among tied candidates with its RNG. On the two larger test sets, where a few points cannot swing the average, agreement is within 0.6%. We read the gap as sample noise, not a bug.

The stump rows show what AdaBoost starts from: a single split reaches 0.8850 on Breast Cancer but only 0.1995 macro-F1 on Covertypes, where with seven classes it predicts essentially one label.

B. AdaBoost scaling

Fig. 1 plots training and test error against boosting rounds. Training error falls monotonically, as the theory says it must while weak learners keep beating chance. Test error on Breast Cancer reaches its best value (0.0177) well before round 200 and finishes slightly worse (0.0265), and on Covertypes the gap is wider: best 0.3265 against a final 0.4000.

So overfitting exists but is mild and late, matching the long-standing observation that AdaBoost’s test error keeps improving well after training error hits zero because the margins keep widening. The practical reading: the number of rounds is not a knob you must tune carefully on clean data, though the noise experiment below shows it becomes one when labels are corrupted.

C. Random Forest scaling

Both curves in Fig. 2 behave as bagging theory predicts. Test accuracy climbs steeply for 20-30 trees then flattens completely: added trees reduce the $\frac{1-\rho}{T}\sigma^2$ term and cannot touch the $\rho\sigma^2$ floor, so the forest converges rather than overfits. Nothing is gained after roughly 50 trees and nothing is lost by adding 150 more - more trees are never harmful, only wasteful.

The OOB curve is the more interesting one. At 200 trees OOB and test accuracy agree closely on Adult (0.8525 vs

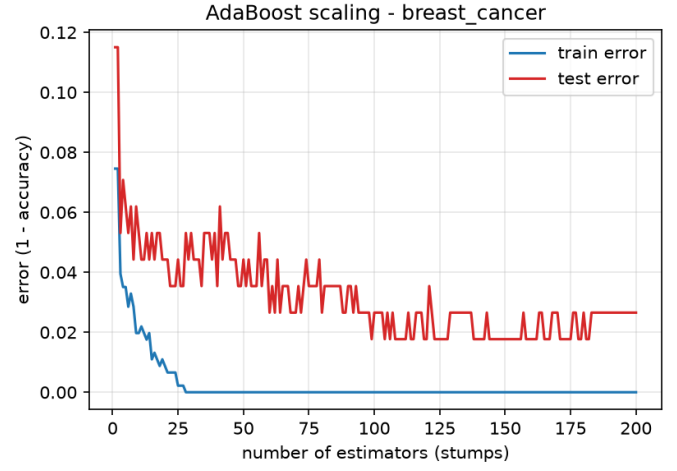


Fig. 1. Experiment 2. AdaBoost training and test error vs. number of stumps on Breast Cancer. Training error is driven towards zero while test error flattens out early, then drifts up slightly - mild overfitting, long after training error stops being informative.

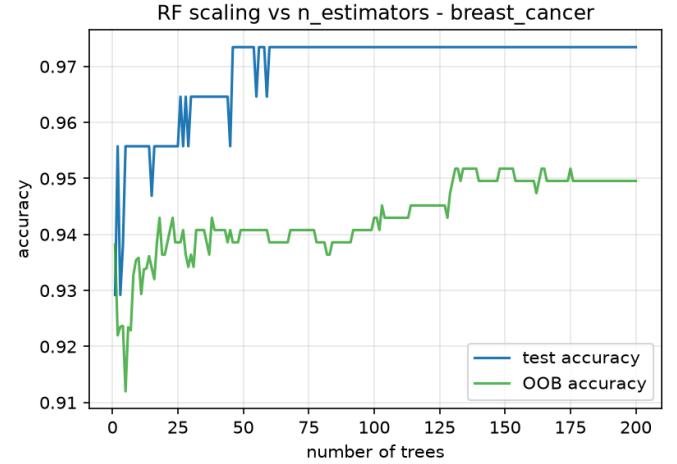


Fig. 2. Experiment 3. Test and OOB accuracy vs. number of trees on Breast Cancer. Both plateau early; OOB tracks test accuracy from a pessimistic direction.

0.8425) and Covertypes (0.7411 vs 0.7440), while OOB sits below test on Breast Cancer (0.9496 vs 0.9735). OOB is pessimistic by construction - each sample is scored only by the $\approx 37\%$ of trees that did not see it, so it measures a smaller forest than the one deployed - and noisy for the first few trees, when many samples have no OOB vote. That it lands within a couple of points of test accuracy on all three datasets is what makes it worth having: a validation estimate that costs nothing and needs no held-out data.

Depth tells the complementary story. Accuracy rises with `max_depth` then flattens rather than degrading, peaking at 0.9735 on Breast Cancer and 0.8488 on Adult. A forest of deep, individually overfitting trees does not itself overfit, because averaging absorbs it - which is the point of bagging, and not what would happen to a single tree.

TABLE III

EXPERIMENT 4. 5-FOLD CV, MEAN $\pm$ STD, 100 ESTIMATORS. SKLEARN'S FOREST IS A REFERENCE POINT, NOT AN IMPLEMENTATION.

Data	Model	Accuracy	Macro-F1	AUC
Breast	Tree	0.9156 $\pm$ 0.020	0.9106 $\pm$ 0.020	0.9136 $\pm$ 0.014
	AdaBoost	0.9754 $\pm$ 0.007	0.9735 $\pm$ 0.007	0.9953 $\pm$ 0.003
	RF (ours)	0.9595 $\pm$ 0.018	0.9565 $\pm$ 0.020	0.9917 $\pm$ 0.006
	RF (skl.)	0.9613 $\pm$ 0.017	0.9585 $\pm$ 0.018	0.9898 $\pm$ 0.010
Adult	Tree	0.8095 $\pm$ 0.024	0.7449 $\pm$ 0.030	0.7444 $\pm$ 0.030
	AdaBoost	0.8575 $\pm$ 0.010	0.7886 $\pm$ 0.017	0.9118 $\pm$ 0.014
	RF (ours)	0.8570 $\pm$ 0.009	0.7964 $\pm$ 0.014	0.9091 $\pm$ 0.011
	RF (skl.)	0.8485 $\pm$ 0.007	0.7854 $\pm$ 0.009	0.8991 $\pm$ 0.010
Cover	Tree	0.6555 $\pm$ 0.015	0.4953 $\pm$ 0.053	0.7216 $\pm$ 0.037
	AdaBoost	0.5885 $\pm$ 0.035	0.3234 $\pm$ 0.078	0.8529 $\pm$ 0.016
	RF (ours)	0.7326 $\pm$ 0.020	0.4632 $\pm$ 0.082	0.9288 $\pm$ 0.010
	RF (skl.)	0.7415 $\pm$ 0.017	0.4989 $\pm$ 0.055	0.9145 $\pm$ 0.015

D. Head-to-head

Table III is the core comparison: 100 estimators each, 5-fold CV.

On the two binary datasets the ensembles are close and both clearly beat a single tree. AdaBoost edges Breast Cancer (0.9754 vs 0.9595), and on Adult the two are indistinguishable - 0.8575 vs 0.8570 accuracy, a gap of five ten-thousandths against a fold-to-fold standard deviation twenty times larger. Reporting that as a win for boosting would be reading noise.

The sklearn row is a sanity check on the forest, and it passes: we land within 0.9 points of the reference everywhere (0.9595 vs 0.9613 on Breast Cancer, 0.8570 vs 0.8485 on Adult, 0.7326 vs 0.7415 on Covertype), ahead on one and behind on two - the scatter expected from two correct implementations differing in tie-breaking and RNG rather than substance. With the single-tree agreement in Table II, we take this as evidence that the ensembles compared here are the algorithms we claim.

Covtype breaks the symmetry and is the most informative row group. AdaBoost (0.5885) loses to a plain decision tree (0.6555) while our forest wins outright (0.7326). The reason is the weak learner: one axis-aligned cut cannot say anything useful about seven classes, and SAMME's α_m is small when every learner sits barely above the $1 - 1/K$ chance threshold. Boosting a hopeless learner yields a weighted average of hopeless learners.

The AUC column complicates this: AdaBoost's Covtype AUC (0.8529) far exceeds the tree's (0.7216) despite worse accuracy. The ensemble ranks classes well but thresholds them badly, which is characteristic of discrete SAMME - the class-wise α sums we normalise are a vote tally, not a calibrated posterior. Macro-F1 shows the other side: every model scores far below its accuracy on Covtype (RF: 0.4632 vs 0.7326) because the rare classes are missed, which accuracy alone would have hidden.

E. Noise robustness

This is where the two philosophies separate, and Table IV makes the split clean.

TABLE IV

EXPERIMENT 5. TEST ACCURACY ON A CLEAN TEST SET AFTER FLIPPING A FRACTION η OF TRAINING LABELS. LAST COLUMN IS THE DROP FROM $\eta = 0$ TO $\eta = 0.20$; SMALLER IS BETTER.

Data	Model	0%	5%	10%	20%	Drop
Bst	Tree	0.929	0.894	0.805	0.673	+0.257
	AdaBoost	0.973	0.947	0.947	0.912	+0.062
	RF	0.973	0.965	0.947	0.965	+0.009
Adult	Tree	0.790	0.766	0.739	0.658	+0.132
	AdaBoost	0.854	0.851	0.845	0.835	+0.019
	RF	0.843	0.851	0.835	0.833	+0.010
Cover	Tree	0.642	0.594	0.586	0.531	+0.112
	AdaBoost	0.618	0.680	0.676	0.670	-0.052
	RF	0.746	0.743	0.731	0.734	+0.012

TABLE V

EXPERIMENT 6. BREIMAN'S 0/1-LOSS DECOMPOSITION OVER 100 BOOTSTRAP REPLICATES, BREAST CANCER. BIAS IS THE ERROR OF THE MAJORITY-VOTE PREDICTION; VARIANCE IS THE AVERAGE DISAGREEMENT WITH IT.

Model	Error	Bias	Variance
Decision Tree	0.0827	0.0265	0.0642
AdaBoost	0.0342	0.0177	0.0235
Random Forest	0.0446	0.0265	0.0234

The single tree collapses - 0.929 to 0.673 on Breast Cancer, 25.7 points - which is what an unpruned, high-variance learner does when you corrupt its training signal: it fits the corruption exactly. Both ensembles are far more robust, but not equally. On Breast Cancer, the cleanest comparison, Random Forest gives up 0.9 points across the range while AdaBoost gives up 6.2, roughly seven times more. Adult repeats the ordering more mildly (1.0 vs 1.9).

The mechanism is equation (4). A flipped label is a point AdaBoost cannot classify correctly, so it is missed every round and its weight multiplied up every round; the ensemble devotes its later rounds to fitting precisely the points carrying no signal. Bagging has no such feedback loop: a corrupted point appears in about 63% of bootstraps, influences those trees only locally, and is outvoted by trees that did not see it. Averaging is indifferent to where the noise came from; re-weighting is not.

Covtype produced the one result we did not expect: AdaBoost *gains* 5.2 points as noise rises, 0.618 to 0.670. We will not over-read a single anomalous cell, but the plausible reading is that noise regularises an ensemble that was already failing (0.5885 clean, per Table III). Its weight distribution here is dominated by majority classes it keeps missing, and randomising labels flattens that distribution, stopping it committing hard to a bad solution. When a model sits below a plain decision tree, disrupting it costs little. This is a symptom of AdaBoost's failure on Covtype, not evidence of robustness.

F. Bias-variance decomposition

Table V is, for our money, the cleanest result in the paper: it confirms the textbook account almost exactly, on real data, from our own implementations.

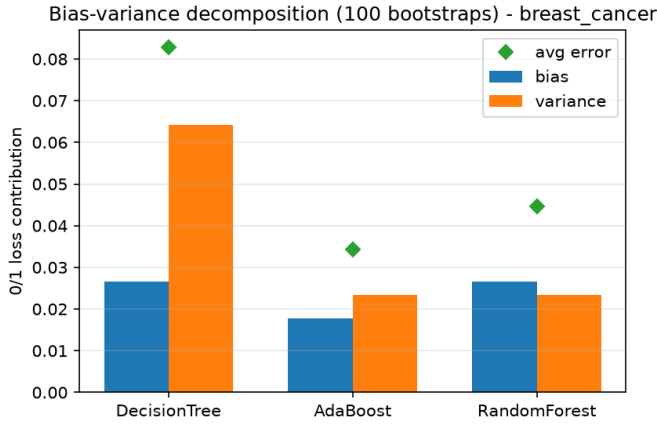


Fig. 3. Experiment 6. Bias and variance per model over 100 bootstrap replicates. Bagging flattens the variance bar and leaves bias untouched; boosting lowers both, and is alone in lowering bias.

Fig. 3 shows the same numbers as bars. Read the Random Forest row against the tree row: the bias is *identical* - 0.0265 in both, not approximately but exactly - while variance falls from 0.0642 to 0.0234, a 64% reduction. That is bagging’s entire claim, isolated. Averaging bootstrap replicates of the same base learner leaves the systematic component of its error alone and removes the sample-to-sample component; a Random Forest can never be less biased than the trees it averages, and here it is not.

AdaBoost alone moves the bias, 0.0265 to 0.0177, a 33% cut, while reducing variance about as much as bagging (0.0235). Lowering both is not a contradiction: each round fits a stump to the current weighted residual, so the ensemble is additive and its capacity grows with M , attacking bias directly, while averaging 50 stumps damps variance as a side effect. This is why AdaBoost wins on Breast Cancer in Table III - it is the only model attacking both terms - and why it loses under noise, since the mechanism that reduces bias is the one that chases mislabelled points.

G. Unsupervised analysis

Table VI collects the pipeline’s output. The PCA column already separates the datasets: Breast Cancer needs 7 of 30 components for 90% variance and its first two carry 63.2%, whereas Adult needs 74 of 104 and its first two carry 7.8%. That is not a defect in the projection but what one-hot encoding does - sparse binary indicators are nearly uncorrelated, so variance spreads evenly across directions and no low-dimensional projection captures much of it. A 2D scatter of Adult is therefore close to meaningless, and we say so rather than reading shapes into it; Fig. 4 shows the Breast Cancer case, where the projection is informative.

K-Means on Breast Cancer reaches ARI 0.671 at $k = 2$: with no access to labels, distance to two centroids recovers most of the malignant/benign distinction. Two caveats. The elbow heuristic (Fig. 5) picks $k = 3$, not the $k = 2$ maximising ARI - the inertia curve is smooth, the knee is a judgement call,

TABLE VI
EXPERIMENT 7. PCA COMPONENTS FOR $\geq 90\%$ VARIANCE, AND BEST ARI FOR EACH CLUSTERING METHOD. k^* AND ε^* ARE THE BEST-ARI SETTINGS.

Dataset	PCs	PC1+2	K-Means ARI	DBSCAN ARI	Noise
Breast Cancer	7	63.2%	0.671 ($k^*=2$)	0.155	25.5%
Adult	74	7.8%	0.187 ($k^*=2$)	0.012	22.3%
Covertypes	36	13.3%	0.099 ($k^*=5$)	0.117	0.9%

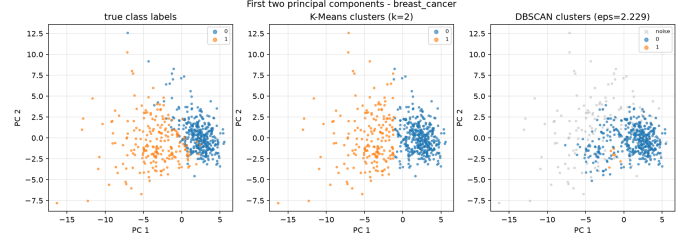


Fig. 4. Experiment 7. First two principal components of Breast Cancer, coloured by true class, K-Means ($k=2$) and DBSCAN. K-Means recovers the malignant/benign split closely (ARI 0.671); DBSCAN sees one connected density region and merges them.

and here the unsupervised heuristic leads slightly astray; we report both rather than quietly using the labels to choose k and calling it an elbow. And ARI on Adult (0.187) and Covertypes (0.099) is close to nothing.

DBSCAN did poorly nearly everywhere, and the k -distance plot (Fig. 6) shows why. At the knee ($\varepsilon = 3.716$) it returns a *single* cluster covering Breast Cancer with 4.9% noise; forcing ε to 2.229 splits it in two but discards a quarter of the data as noise, for ARI 0.155. Neither is good, and the reason is structural: DBSCAN separates clusters only where density genuinely drops between them, and here the classes are adjacent regions of one continuous cloud with no low-density channel between. No ε fixes that - too large merges the classes, too small shatters them into noise, and the sweep shows exactly that trade-off with nothing usable in between. On Adult it is essentially at chance (best ARI 0.012, negative at four of six ε values), the curse of dimensionality doing what it does to density estimates. Covertypes is the one dataset where DBSCAN edges K-Means (0.117 vs 0.099), plausibly because its clusters are neither spherical nor equally sized - the case K-Means handles worst - though both numbers are low enough that the comparison is thin.

The payoff is the correlation with the supervised results, which is strikingly monotone. Ranking by K-Means ARI gives Breast Cancer (0.671), Adult (0.187), Covertypes (0.099); ranking by best ensemble accuracy in Table III gives Breast Cancer (0.975), Adult (0.858), Covertypes (0.733) - the same order. Where classes are recoverable without labels, they are easy to recover with them.



Fig. 5. Experiment 7. K-Means inertia vs. k on Breast Cancer. The detected knee is $k = 3$, while $k = 2$ maximises ARI against the true labels - an example of the elbow heuristic being suggestive rather than decisive.

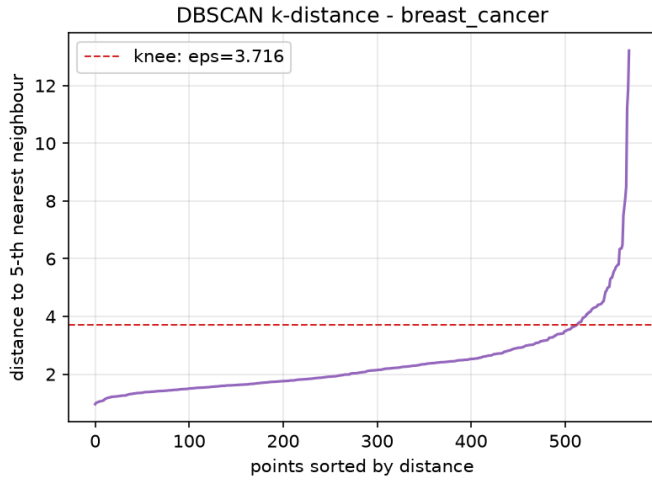


Fig. 6. Experiment 7. Sorted distance to the 5th nearest neighbour, Breast Cancer. The knee at $\epsilon \approx 3.716$ is gentle, which is itself the finding: there is no sharp density gap for DBSCAN to exploit.

VI. DISCUSSION

A. Answering the central question

Our evidence says the honest answer is narrower than the question implies.

Boosting wins when the base learner’s bias is the binding constraint and the labels are trustworthy. Breast Cancer is that case: clean, balanced, low-dimensional enough that a stump is informative, and AdaBoost takes it (0.9754 vs 0.9595). Table V says why - AdaBoost is the only model here that reduced bias. When variance is already handled and what remains is systematic error, only boosting has a mechanism for it.

Bagging wins when variance dominates, when labels are noisy, or when the weak learner is too weak. All three appear in our results: RF loses 0.9 points to AdaBoost’s 6.2 under 20% noise, and on Covertypes, where a stump cannot

address seven classes, AdaBoost (0.5885) finishes below a single tree while RF wins (0.7326).

Most of the time it does not matter much. On Adult the two differ by 0.0005 against a standard deviation of 0.01. This is worth stating plainly, because a study of this shape invites you to declare a winner from a difference the folds cannot resolve. The defensible claim is not that one method is better but that they fail differently, and predictably from Table V.

If forced to pick a default: Random Forest - not because it won more often, since it did not, but because its failure mode is graceful. It cannot overfit as trees are added, its bad case is mediocrity rather than collapse, and OOB gives a free honest estimate of how it is doing. AdaBoost is the better instrument when the labels are clean and bias is what hurts.

B. Dataset characteristics

Dimensionality hurt the unsupervised half far more than the supervised half. Adult’s 104 one-hot features left DBSCAN at chance and needed 74 PCs for 90% variance, yet both ensembles reached about 0.857. Trees do implicit feature selection - they never split on a useless indicator - while distance-based methods must use every dimension given.

Class imbalance was the most instructive. Covertypes’s 0.47% minority is invisible in accuracy and obvious in macro-F1, where RF drops from 0.7326 accuracy to 0.4632. Reporting without resampling is the right default for a comparative study, but it means our Covertypes numbers describe models that largely ignore the rare classes. Class weights map naturally onto the `sample_weight` interface our tree already exposes, which makes them a better fit here than SMOTE - interpolating synthetic neighbours between one-hot indicators produces points corresponding to no real category assignment. Quantifying that trade-off is the most obvious thing missing from this study.

Noise produced the clearest separation, for no mysterious reason: equation (4) cannot tell a hard example from a wrong one.

C. Do clusters predict classifier performance?

Yes, on our three datasets, and the ordering is exact. ARI measures whether class structure lies along the directions of greatest variance, which is what unsupervised methods can see; when it does, the problem is easy for everything. When it does not - Adult, where classes depend on interactions between sparse indicators rather than dominant variance directions - clustering finds nothing while the ensembles still reach 0.857, because a tree carves out structure no centroid distance reveals. So the relationship is one-directional: high ARI predicts easy classification, but low ARI does not predict hard classification. Three datasets is also three points, which cannot support more than a suggestive trend.

D. Limitations

Subsampling is the main one: capping Adult at 8,000 rows and the sweeps at 2,000 depresses every absolute number, and the caps were set by our implementation’s speed rather than by

the science. Model rankings are unaffected, since all models saw the same rows, but these are not the best achievable accuracies here. The bias-variance decomposition runs on one dataset, as the brief specifies, so its clean confirmation should not be over-generalised. Our AdaBoost is discrete SAMME; SAMME.R would likely improve the Covertypes probabilities given the AUC/accuracy gap. And the Covertypes noise anomaly deserves proper investigation rather than the plausible story we offered.

VII. CONCLUSION

We implemented a CART decision tree, AdaBoost with stumps, and a Random Forest from scratch, verified them against sklearn (within 0.6% on the two larger datasets), and used them to test when boosting beats bagging.

The bias-variance decomposition gave the sharpest answer: bagging left the base learner's bias exactly unchanged while cutting variance by 64%, and AdaBoost was the only method to reduce bias. Everything else follows. Boosting won on clean, balanced Breast Cancer, where bias was what remained to remove. Bagging won decisively under label noise (0.9 points lost against AdaBoost's 6.2 at $\eta = 0.20$) and on multi-class Covertypes, where a stump is too weak to boost and AdaBoost finished below a single tree. On Adult the two were indistinguishable. The unsupervised pipeline ranked the datasets by ARI in the same order the classifiers ranked them by accuracy, suggesting the two views agree about which problems are hard.

The most useful extension is the imbalance treatment we implemented but did not evaluate: measuring the accuracy/macro-F1 trade-off from class weights on Covertypes would address this study's weakest results directly. Beyond that, Gradient Boosting would separate AdaBoost's exponential loss from boosting in general, and t-SNE would test whether Adult's class structure is genuinely absent from low dimensions or merely invisible to a linear projection.

TOOLS AND ACKNOWLEDGEMENTS

In line with the course's academic integrity policy, we disclose that an AI assistant Claude Sonnet 4.5 was used during this project. It helped to write: test cases, format plots and report; 'fit', 'compute oob score' in random forest; 'fit' in adaboost, 'best split' and 'build' in decision tree, dbscan, kmeans

REFERENCES

- [1] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. [Online]. Available: <https://hastie.su.domains/ElemStatLearn/>
- [2] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81-106, 1986.
- [3] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119-139, 1997.
- [4] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123-140, 1996.
- [5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [6] K. Pearson, "On lines and planes of closest fit to systems of points in space," *Philosophical Magazine*, vol. 2, no. 11, pp. 559-572, 1901.
- [7] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129-137, 1982.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd ACM SIGKDD*, 1996, pp. 226-231.